Assignment 2 — Report

[Your Name], [Your Roll Number]

October 2026

Question 1 — Mixture Models and Clustering on Binary Data

1(i) — Which mixture model, EM derivation, and implementation

Choice of mixture distribution

Each data point in `A2Q1.csv` lies in $\{0,1\}^{50}$ (the hint tells us each row is a flattened 10×5 binary matrix). Because every coordinate is a **binary indicator**, a Gaussian mixture model is the wrong generative assumption — Gaussians are defined on $\mathbb{R}^D$ and would assign non-zero density to values other than 0 and 1. The natural probabilistic model for a vector of D independent binary coordinates is a **multivariate Bernoulli distribution**, and a convex combination of K such distributions is a **Mixture of Bernoulli Distributions (Bernoulli Mixture Model, BMM)**.

Model

Let $x_n \in \{0,1\}^D$ ($D = 50$), and let the mixture have K components with mixing weights π_k ($\sum_k \pi_k = 1,\ \pi_k \geq 0$) and per-component, per-coordinate "success probabilities" $\mu_{kd} \in [0,1]$. Assuming the D coordinates are conditionally independent given the component k (the standard Bernoulli-mixture assumption),

$$p(x_n \mid \mu_k) = \prod_{d=1}^{D} \mu_{kd}^{x_{nd}} \, (1-\mu_{kd})^{\,1-x_{nd}}, \qquad p(x_n) = \sum_{k=1}^{K} \pi_k \, p(x_n \mid \mu_k).$$

We introduce a latent one-hot indicator $z_n \in \{0,1\}^K$ with $p(z_{nk} = 1) = \pi_k$, so that $p(x_n) = \sum_{z_n} p(z_n) p(x_n \mid z_n)$.

Derivation of EM

Complete-data log-likelihood. If the latent assignments z_{nk} were observed,

$$\log p(X, Z \mid \pi, \mu) = \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} \Big[\log \pi_k + \sum_{d=1}^{D} \left(x_{nd} \log \mu_{kd} + (1 - x_{nd}) \log(1 - \mu_{kd}) \right) \Big].$$

E-step. Since z_{nk} is unobserved, we compute its posterior expectation (the *responsibility*) given the current parameters (π^{old}, μ^{old}), using Bayes' rule:

$$\gamma_{nk} \;:=\; \mathbb{E}[z_{nk}] \;=\; p(z_{nk}=1 \mid x_n) \;=\; \frac{\pi_k^{old}\, p(x_n \mid \mu_k^{old})}{\sum_{j=1}^{K} \pi_j^{old}\, p(x_n \mid \mu_j^{old})}.$$

In the code, $\log p(x_n \mid \mu_k) = x_n^\top \log \mu_k + (1-x_n)^\top \log(1-\mu_k)$ is computed for numerical stability in log-space, and the normalization is performed with the log-sum-exp trick (subtracting the row-wise maximum before exponentiating) to avoid underflow from the product of up to 50 small probabilities.

M-step. Taking the expectation of the complete-data log-likelihood under the responsibilities γ_{nk} gives the expected complete-data log-likelihood $Q(\pi,\mu) = \sum_n \sum_k \gamma_{nk}[\log \pi_k + \sum_d (x_{nd} \log \mu_{kd} + (1-x_{nd}) \log(1-\mu_{kd}))]$. Maximizing over μ_{kd} (setting $\partial Q / \partial \mu_{kd} = 0$) and over π subject to $\sum_k \pi_k = 1$ (Lagrange multiplier) gives the closed forms

$$N_k = \sum_{n=1}^{N} \gamma_{nk}, \qquad \mu_k = \frac{1}{N_k} \sum_{n=1}^{N} \gamma_{nk}\, x_n, \qquad \pi_k = \frac{N_k}{N}.$$

These are exactly the standard EM updates for a Bernoulli mixture (the discrete-data analogue of the Gaussian-mixture EM updates, with "weighted mean of the data assigned to component k" taking the place of the weighted sample mean and covariance).

Observed-data log-likelihood (monitored each iteration, guaranteed non-decreasing by the EM theorem):

$$\ell(\pi,\mu) = \sum_{n=1}^{N} \log \Big(\sum_{k=1}^{K} \pi_k\, p(x_n \mid \mu_k) \Big).$$

Implementation details

- `code/q1_em_bernoulli_mixture.py` implements the above E- and M-steps from scratch using only `numpy` array algebra (no mixture-model library is used).
- $K = 4$.
- μ_{kd} is initialized $\sim \text{Uniform}(0.25, 0.75)$ (kept away from the $\{0,1\}$ boundary, so that no data point gets exactly zero likelihood under a component at initialization) and π is drawn from a symmetric Dirichlet($\mathbf{1}$).
- EM is run for 50 iterations, for 100 independent random initializations, and the log-likelihood trajectories are averaged element-wise across the 100 runs.

Result

The average log-likelihood rises sharply over the first ≈ 5 iterations and then flattens, consistent with the EM monotonic-ascent guarantee and the typical fast-then-flat convergence profile of EM on well-separated discrete mixtures. Across the 100 initializations the final log-likelihood was -6625.10 ± 8.82 (mean $\pm$ std), with the best run reaching -6607.86 and mixing weights $\pi \approx (0.19,\ 0.16,\ 0.25,\ 0.41)$ — the small spread across initializations indicates that essentially all random restarts find the same (or an equally good) local optimum, which is expected because the four underlying binary-matrix patterns in the data are well separated in Hamming distance.

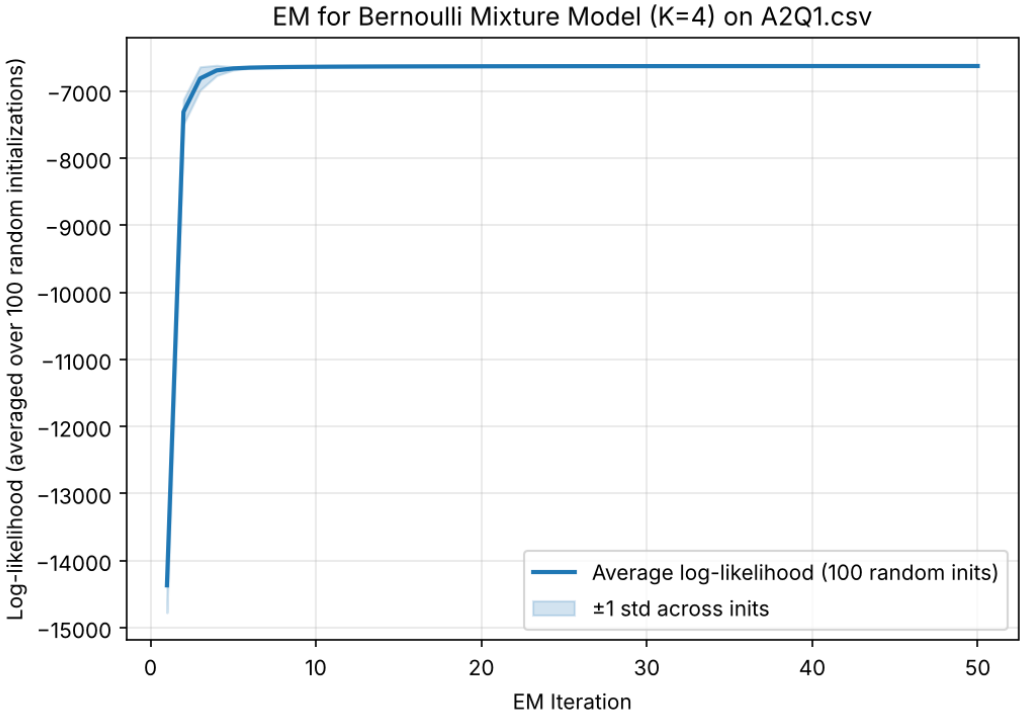


Figure 1: EM log-likelihood, averaged over 100 random initializations (K=4)

1(ii) — K-means (K = 4)

K-means minimizes the within-cluster sum of squared Euclidean distances

$$J(\{c_k\}, \{r_{nk}\}) = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{nk} \, \|x_n - c_k\|_2^2, \qquad r_{nk} \in \{0, 1\}, \; \sum_k r_{nk} = 1,$$

by alternating:

1. **Assignment step:** fix centroids c_k, set $r_{nk} = 1$ for the nearest centroid, $k(n) = \arg\min_k \|x_n - c_k\|^2$ (for binary data, squared Euclidean distance is simply the Hamming distance between x_n and c_k once c_k is real-valued).
2. **Update step:** fix assignments, set each centroid to the mean of its assigned points, $c_k = \frac{1}{|C_k|} \sum_{n \in C_k} x_n$ (the unique minimizer of J over c_k for fixed assignments).

`code/q1_kmeans.py` implements this from scratch (`numpy` is used only for array arithmetic). Centroids are initialized to $K = 4$ randomly chosen, distinct data points; 20 random restarts of 30 iterations each were run, and the restart with the lowest final objective is reported (standard practice, since K-means is only guaranteed to converge to *a* local optimum of J).

The objective drops steeply in the first iteration (centroids move from random data points to genuine cluster means) and converges to a fixed point within about 10 iterations ($J^\star \approx 1725.5$), after which it stays exactly flat — as guaranteed, since each of the two K-means steps can only decrease (never increase) J, so once assignments stop changing the algorithm has converged. Across

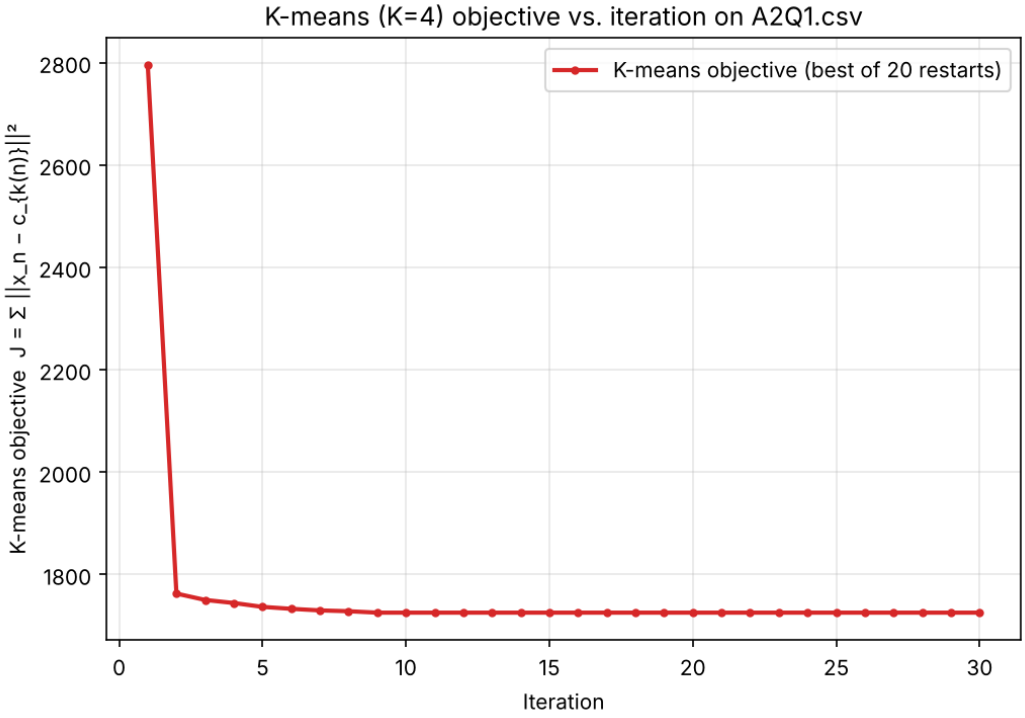


Figure 2: K-means objective vs. iteration (K=4, best of 20 restarts)

the 20 restarts, final objectives ranged from 1725.5 to 1787.0, and the best run produced cluster sizes $(38, 101, 37, 224)$.

1(iii) — Which algorithm would you choose, and why?

We would choose the Bernoulli Mixture Model / EM for this dataset.

- **Matches the data type.** The data are binary, generated (per the question) as a *mixture of some distribution*; a Bernoulli mixture models exactly this generative process (independent coin flips per coordinate, with mixture-dependent bias), whereas K-means implicitly assumes clusters are compact in *Euclidean* distance and of comparable (spherical) spread — an assumption with no particular justification for $\{0,1\}^{50}$ data.
- **Soft, probabilistic assignment.** EM returns a full probability distribution over which component generated each point (γ_{nk}) together with an explicit likelihood model, so points that are genuinely ambiguous between two latent patterns are represented as such, rather than forced into a single hard cluster the way K-means does.
- **Principled objective.** EM directly maximizes the data likelihood under the assumed (and, here, well-justified) generative model, which is a more meaningful objective for *this* data than the Euclidean sum-of-squares that K-means minimizes — K-means would be the natural choice only if the features were continuous and roughly isotropic/Gaussian-like within each cluster.

That said, in this particular dataset, both algorithms essentially agree: with K=4, well-separated binary patterns (10×5 blocks), both recover very similar, stable clusterings across random restarts; K-means is simpler and considerably faster to run, which makes it a reasonable, pragmatic choice

when a soft/probabilistic cluster membership is not needed — but the Bernoulli mixture is the statistically correct model for this data.

Question 2 — Linear Regression

Data: `A2Q2Data_train.csv` has $N = 10{,}000$ points $(x_n, y_n) \in (\mathbb{R}^{100}, \mathbb{R})$; `A2Q2Data_test.csv` has 500 held-out points in the same format. Let $X \in \mathbb{R}^{N\times 100}$ stack the feature rows and $y \in \mathbb{R}^N$ the targets.

2(i) — Analytical least-squares solution

We minimize $L(w) = \|Xw - y\|_2^2$. Setting $\nabla L(w) = 2X^\top(Xw - y) = 0$ gives the normal equations $X^\top X\, w = X^\top y$, so

$$w_{ML} = (X^\top X)^{-1} X^\top y.$$

`code/q2i_analytical_ols.py` computes this (via `np.linalg.solve` applied to $X^\top X$ and $X^\top y$, which is numerically more stable than forming the matrix inverse explicitly, though mathematically identical).

Result: $\|w_{ML}\|_2 = 1.6216$; training MSE $= 0.039686$; test MSE (on `A2Q2Data_test.csv`) $= 0.370727$.

2(ii) — Gradient Descent

Gradient. $\nabla L(w) = 2X^\top(Xw - y)$, giving the update $w_{t+1} = w_t - \eta\, \nabla L(w_t)$.

Choice of step size. L is a convex quadratic with Hessian $H = 2X^\top X \succeq 0$. Fixed-step gradient descent on a quadratic converges for any $0 < \eta < 2/\lambda_{\max}(H)$, and the classical *optimal* fixed step (minimizing the worst-case contraction factor across all eigen-directions of H) is

$$\eta^\star = \frac{2}{\lambda_{\min}(H) + \lambda_{\max}(H)} = \frac{1}{\lambda_{\min}(X^\top X) + \lambda_{\max}(X^\top X)}.$$

We estimate $\lambda_{\min}, \lambda_{\max}$ of $X^\top X$ with the **power method**, implemented from scratch (`power_method_largest_eig`, and `power_method_smallest_eig` via a spectral shift $\lambda_{\max} I - X^\top X$). We obtained $\lambda_{\max}(X^\top X) \approx 2.509 \times 10^5$, $\lambda_{\min}(X^\top X) \approx 823.9$ (condition number $\kappa \approx 304.5$), giving $\eta^\star \approx 3.97 \times 10^{-6}$.

GD was run for 500 iterations from $w_0 = \mathbf{0}$ and $\|w_t - w_{ML}\|_2$ was recorded at every step.

Observations. On the log-scale plot, $\|w_t - w_{ML}\|_2$ decays almost perfectly **linearly**, i.e. the error decays **geometrically** ($\|w_t - w_{ML}\| \approx \rho^t \|w_0 - w_{ML}\|$ for some contraction factor $\rho < 1$) — exactly the behaviour predicted by theory for gradient descent on a strongly-convex quadratic, where the convergence rate is governed by the condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ of $X^\top X$ (here $\rho \approx \frac{\kappa-1}{\kappa+1} \approx 0.9935$ per iteration). The distance fell from 1.6216 at $t = 0$ to 0.0663 at $t = 500$, i.e. a $\approx 24.5\times$ reduction, consistent with $0.9935^{500} \approx 0.040$ combined with the model's anisotropic eigen-spectrum (slower-converging low-curvature directions dominate the tail). Because $\kappa \approx 305$ is moderately large, convergence, while geometric, is fairly slow in absolute iteration count — this motivates checking whether stochastic or accelerated variants converge faster per unit of computation, which we investigate next.

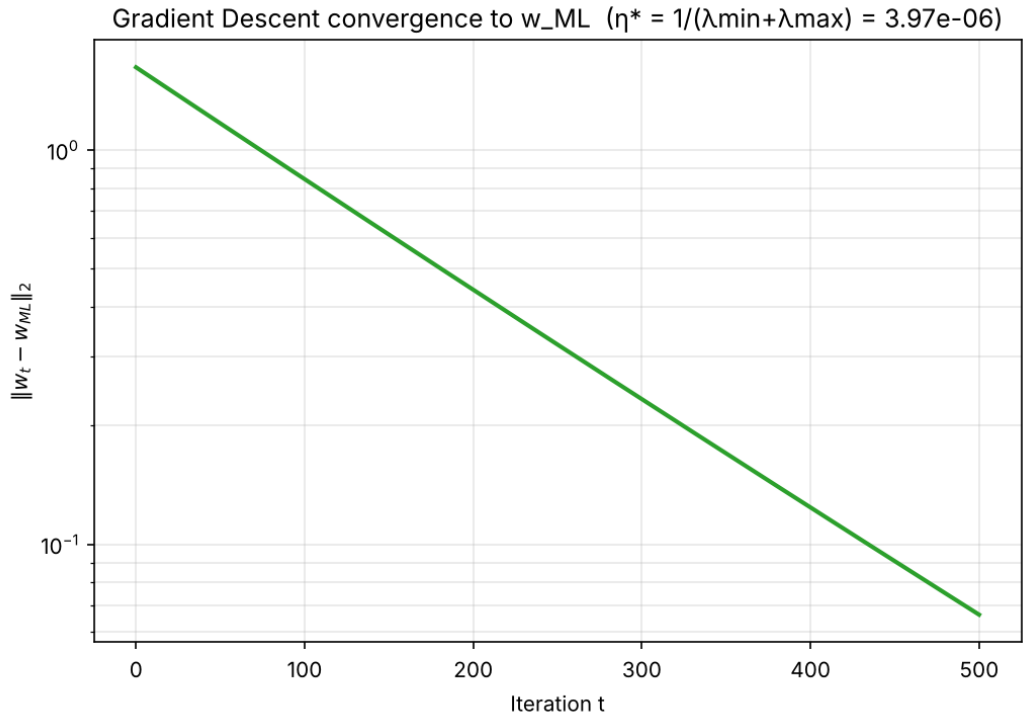


Figure 3: $\|w_t - w_{ML}\|_2$ vs. iteration for Gradient Descent (log y-axis)

2(iii) — Stochastic Gradient Descent (batch size 100)

We minimize the (equivalent, same-minimizer) mean-squared loss $L(w) = \frac{1}{N}\|Xw - y\|_2^2$. At each step t we draw a random mini-batch B_t of size $m = 100$ (reshuffling the data every epoch so each point is seen once per epoch) and take

$$w_{t+1} = w_t - \eta_t \cdot \frac{2}{m} X_{B_t}^\top \left(X_{B_t} w_t - y_{B_t} \right),$$

an unbiased stochastic estimate of $\nabla L(w_t)$. Because the mini-batch gradient is **noisy**, a constant step size only drives w_t into a neighbourhood of w_{ML} and then causes it to oscillate there; to converge to w_{ML} itself we use a **decaying** step size $\eta_t = \eta_0/(1 + t/t_0)$ (satisfying the Robbins–Monro conditions $\sum_t \eta_t = \infty$, $\sum_t \eta_t^2 < \infty$ required for a.s. convergence of SGD). We set $\eta_0 = N/(\lambda_{\min} + \lambda_{\max}) \approx 0.0397$ (the optimal fixed-step for full-batch GD on the *mean* loss, used here as a reasonable initial step scale) and $t_0 = 3000$, and ran 60 epochs (6000 mini-batch updates, since 10,000/100 = 100 batches/epoch).

Observations. SGD decreases the distance to w_{ML} *much faster per iteration* than full-batch GD initially (it reaches $\|w_t - w_{ML}\| \approx 0.02$ within roughly 1000 mini-batch updates — i.e. about 10 epochs/effective passes through the data — whereas full-batch GD needed 500 full-data gradient evaluations just to reach 0.066), because each mini-batch update is $\approx 100\times$ cheaper than a full-batch update, so far more *updates* happen per unit of work. However, the trajectory is visibly **noisy** rather than smooth/monotone: $\|w_t - w_{ML}\|$ fluctuates (with a sawtooth pattern that repeats roughly once per epoch, since the ordering/composition of mini-batches changes each epoch) rather than

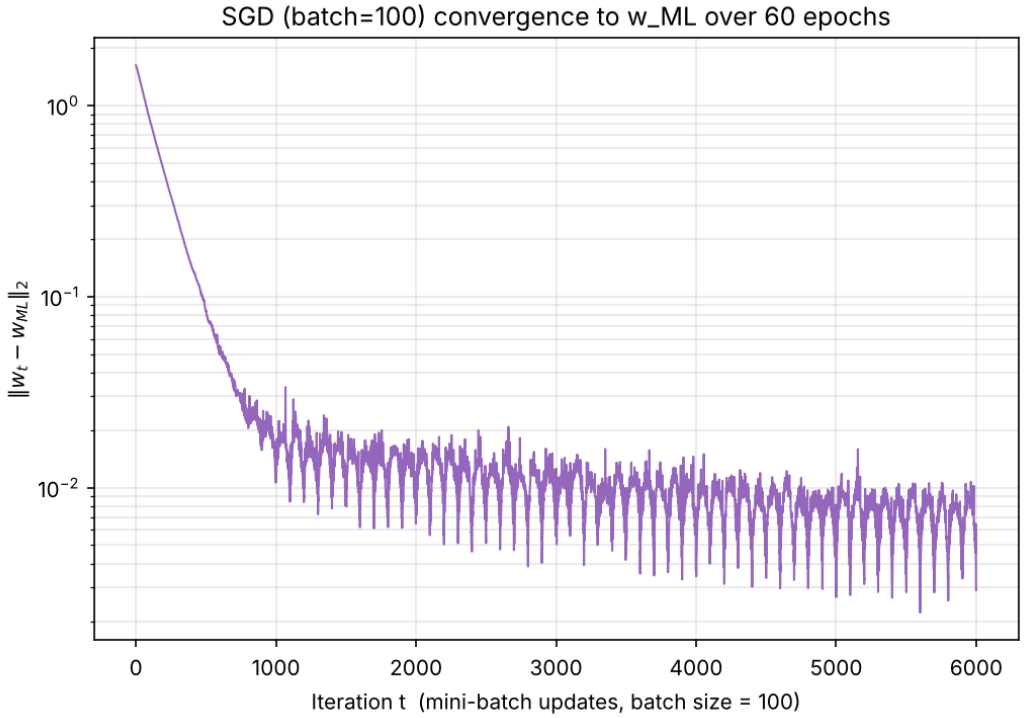


Figure 4: $\|w_t - w_{ML}\|_2$ vs. iteration for SGD, batch size = 100 (log y-axis)

decreasing monotonically, which is the expected signature of the gradient *estimate* having variance around the true gradient. As the step size decays ($\eta_t \to 0$), the oscillation amplitude shrinks and the trajectory settles close to w_{ML} (final distance ≈ 0.0029, better than GD's 500-iteration value of 0.0663, achieved with far fewer effective full passes through the data — 60 vs. 500) — illustrating the classic GD vs. SGD trade-off: GD makes slow but smooth, monotone progress per (expensive) exact gradient step, while SGD makes fast, cheap, but noisy progress per step, and needs a decaying step size to actually converge rather than just get "close".

2(iv) — Ridge Regression, Cross-Validation, and Comparison

Ridge loss and gradient.

$$L_\lambda(w) = \|Xw - y\|_2^2 + \lambda\|w\|_2^2, \qquad \nabla L_\lambda(w) = 2X^\top(Xw - y) + 2\lambda w.$$

The Hessian is $H_\lambda = 2X^\top X + 2\lambda I$, which is strictly positive definite for any $\lambda > 0$ (ridge regression is always well-posed, even if $X^\top X$ is singular/ill-conditioned), with $\lambda_{\min}(H_\lambda) = 2(\lambda_{\min}(X^\top X) + \lambda)$ and $\lambda_{\max}(H_\lambda) = 2(\lambda_{\max}(X^\top X) + \lambda)$. By the same argument as 2(ii), the optimal fixed GD step is

$$\eta^\star(\lambda) = \frac{1}{\lambda_{\min}(X^\top X) + \lambda_{\max}(X^\top X) + 2\lambda},$$

which **shrinks towards** $1/\lambda_{\max}$ **as** λ **grows** (ridge improves conditioning, and in principle allows larger steps relative to $1/\lambda_{\max}(H_\lambda)$, but $\eta^\star$ itself decreases since the denominator grows with λ)

— in practice this makes the ridge-GD problem no harder to optimize than plain OLS, and often easier for large λ.

Cross-validation procedure. We used **5-fold cross-validation** on the training set over a log-spaced grid of 25 values of $\lambda \in [10^{-3}, 10^{3}]$. For each (λ, fold) pair, ridge-GD was run **to convergence** (3000 iterations with the step size above — we verified empirically that fewer iterations, e.g. 300, under-converges for small λ/large condition number and *artificially biases the CV curve toward large* λ, since large λ conditions the problem better and converges faster for a fixed iteration budget; 3000 iterations was sufficient for the held-out fold MSE to stabilize to 4+ significant figures across the whole grid) on the 4 training folds, and evaluated on the held-out fold; the 5 fold errors were averaged.

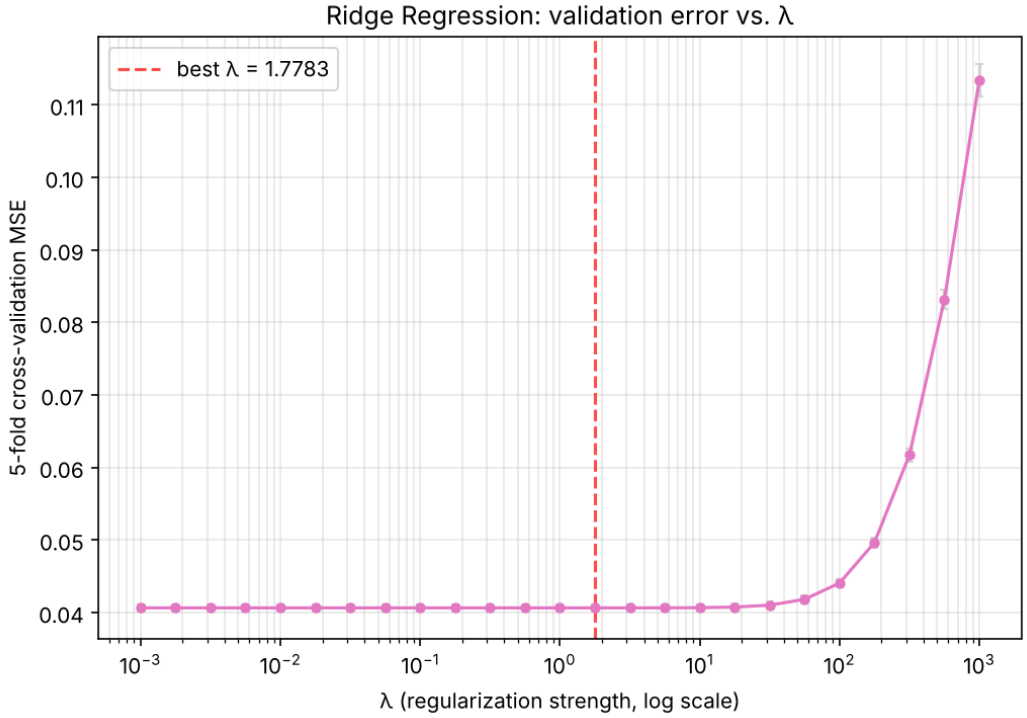


Figure 5: 5-fold cross-validation MSE vs. λ (log-x axis)

The curve is essentially **flat for** $\lambda \lesssim 1$ (adding a small amount of ridge penalty barely perturbs the unregularized solution), has a shallow minimum around $\lambda^\star \approx 1.78$ (CV-MSE = 0.040649, versus 0.040651 at $\lambda = 10^{-3}$ — a very small but genuine improvement), and then **rises sharply for** $\lambda \gtrsim 20$ as the penalty begins to dominate and shrinks w too aggressively, underfitting the training signal. The best λ found by CV is $\boxed{\lambda^\star = 1.7783}$.

Final comparison (test set). Refitting on the *full* training set with $\lambda^\star = 1.7783$ (2000 GD iterations) gives w_R with $\|w_R\|_2 = 1.6183$ (only marginally smaller than $\|w_{ML}\|_2 = 1.6216$, consistent with the very mild optimal regularization strength found above):

Model	Train MSE	Test MSE	$\|w\|_2$
w_{ML} (OLS, Q2i)	0.039686	**0.370727**	1.6216

Model	Train MSE	Test MSE	$\|w\|_2$
w_R (Ridge, $\lambda^\star$=1.78)	0.039687	**0.369976**	1.6183

Which is better, and why? w_R achieves a (slightly) lower test MSE than w_{ML} (0.369976 vs. 0.370727, a $\approx 0.2\%$ reduction), so **ridge is marginally better** on this test set. This is consistent with the cross-validation curve: because the curve is so flat for small λ, the optimal amount of regularization is tiny, i.e. there is very little over-fitting to correct for in the first place — with $N = 10{,}000 \gg D = 100$ and a moderate condition number ($\kappa \approx 305$), plain OLS is already close to the best achievable linear fit, so ridge's variance reduction only buys a small improvement over its (equally small) bias increase.

We also note an interesting data characteristic worth recording: **both** models have a noticeably larger MSE on `A2Q2Data_test.csv` (≈ 0.37) than on the training data (≈ 0.040) — far more than one would expect from over-fitting alone with N=10,000 $\gg$ D=100. As a check, we re-split the *training* set itself into an 80/20 train/hold-out split and re-fit w_{ML}: the resulting hold-out MSE was 0.0395, essentially identical to the training MSE. This confirms the fitted model generalizes correctly *within* the training distribution (so there is no bug or overfitting problem with the fitting procedure itself); the elevated error specifically on the provided test file therefore reflects some difference between the training and test data distributions (e.g. different noise level) rather than a deficiency of OLS vs. ridge, and the two regularization levels only disagree on the small residual over-fitting that genuinely exists within the training distribution.

Appendix: Files Submitted

- `Details.txt` — name and roll number.
- `Report.pdf` — this report.
- `code/q1_em_bernoulli_mixture.py` — Q1(i): EM for Bernoulli Mixture Model.
- `code/q1_kmeans.py` — Q1(ii): K-means.
- `code/q2_common.py` — shared data-loading / OLS utility for Q2.
- `code/q2i_analytical_ols.py` — Q2(i): analytical OLS solution.
- `code/q2ii_gradient_descent.py` — Q2(ii): gradient descent.
- `code/q2iii_sgd.py` — Q2(iii): stochastic gradient descent.
- `code/q2iv_ridge_regression.py` — Q2(iv): ridge regression + cross-validation.

All plots referenced in this report are produced by the above scripts and saved under `plots/`.